HVE Core challenge

From product intent to reviewed code

What HVE is

Hypervelocity Engineering is a structured engineering framework for GitHub Copilot.

It gives agent work defined roles, standards, human review gates, and saved phase files.

Command names change between versions. **Check the HVE Core README for your version.**

RPI in one slide

Research → **P**lan → **I**mplement → **R**eview, then Follow-up.

Phase	Prompt	Writes
Research	`/rpi-research`	What exists, limits, rejected options
Plan	`/rpi-plan`	Tasks, acceptance criteria, checks, critique
Implement	`/rpi-implement`	Code and a changes log
Review	`/rpi-review`	Per-criterion verdicts and a decision

Follow-up sends defects to Implement, decision gaps to Plan, and evidence gaps to Research.

Four component types

Component	Job
Agents	Own roles and multi-step protocols
Prompts	Start repeatable workflows
Instructions	Apply standards by file pattern
Tracking artifacts	Carry state between phases

File as interface

Each phase writes a Markdown file. The next phase reads that file, not the chat.

```
.copilot-tracking/research/<date>/<task>-research.md
  → plans/<date>/<task>-plan.md  (+ critique under reviews/plans/)
  → changes/<date>/<task>-changes.md
  → reviews/logs/<date>/<task>-review.md
```

You can read, diff, and edit a file, then attach it to a fresh chat and carry on.

Paths vary by version.

Research before code

Research answers:

- Which files own the behavior?
- What constraints already exist?
- Which alternatives were rejected?
- What check could disprove the approach?

Output: `.copilot-tracking/research/`

Plan before implementation

Plan maps criteria to files, checks, dependencies, and human gates.

Output:

```
.copilot-tracking/plans/
```

Implement and Review

Implement follows the approved plan and writes a changes log.

Review checks whether the result meets the request, code quality standards, tests, and instructions.

The human still decides: approve, request changes, or pause.

Direct and combined RPI

	Direct prompts	Combined `/rpi task="..."`
You type	One prompt per phase	One prompt for the task
Approval	You approve each phase	Depends on the mode you pick
Output	One file per phase	The same kinds of files, one task name
After Review	You route defects	Follow-up routes them
Use it when	Learning, demos, approvals	The team trusts the handoffs

Both write the same files. Combined only changes who drives.

Optional: HVE customization is additive

```
.github/copilot-instructions.md
.github/instructions/*.instructions.md
.github/agents/*.agent.md
```

Project context reaches all agents. Custom agents add repository-owned roles and output contracts.

An instruction loads by `applyTo` glob, set in its frontmatter:

```
applyTo: "src/**/*.js,tests/**/*.test.js"
```

Copilot loads it only when working on a matching file.

Check that it's active

Creating a file doesn't mean HVE used it.

File	Check
Instructions	Edit a matching file, ask a question. The file shows in References. A non-matching file doesn't load it
Agent	It appears in the agent picker (`/agent` in the CLI). Its answer follows your format
Prompt	Typing `/name` shows it in the picker, and it runs

Then find one plan, code, or review change it caused.

Optional: HVE starts before engineering

```
Business idea
  → BRD Builder / Product Manager Advisor / PRD Builder
  → Agile Coach
  → RPI
  → reviewed code
```

BRD: business requirements, why the business wants this. **PRD**: product requirements, what the product must do. `@BRD Builder` and `@PRD Builder` write them.

Product Manager Advisor tests value and assumptions. Agile Coach turns intent into a user story and acceptance criteria. The lab starts from a supplied product intent.

Challenge sequence

1. Run direct Research and Plan prompts.
2. Run Implement and Review.
3. Compare the result with combined `/rpi`.

Optional, if time allows, before Implement: add repository context, an `applyTo` instruction, and a custom reviewer, and use Product Manager Advisor and Agile Coach.

Build the endpoint

Deliver `POST /api/decisions` within the supplied product intent and API contract.
Stay inside the contract.

The contract

Input	Rule
`title`	Required non-empty string; duplicate check ignores case
`context`	Required non-empty string
`decision`	Required non-empty string
`status`	`proposed`, `accepted`, or `superseded`

Generate an ID and UTC timestamp. Keep state in memory.

Package tested source in a non-root runtime image with production dependencies and a health check.

Fast checks that find weak work

1. Send a required field as whitespace or a number.
2. Create the same trimmed title with different case.
3. Send malformed JSON.
4. Inspect the exact error envelope.
5. Check which stage supplies source to the runtime image.

These checks test invalid inputs and container build boundaries.

Keep the handoff short

`challenge-handoff.md` holds the four file paths, your decision, open risks, and the next action. Don't copy phase text or command output into it.